Exp: 29 Priority-Based Interrupt Handling

Program:

```
// Code your testbench here.
// Uncomment the next line for SystemC modules.
// #include <systemc>
#include <systemc.h>
SC_MODULE(priority_interrupt) {
sc_event low_int, high_int;
void low_ISR() {
while(true) {
wait(low_int);
cout << "Low Priority Interrupt at "
<< sc_time_stamp() << endl;
wait(3, SC_SEC);
}
}
void high_ISR() {
while(true) {
wait(high_int);
cout << "High Priority Interrupt at "
<< sc_time_stamp() << endl;
wait(1, SC_SEC);
}
}
void generator() {
wait(1, SC_SEC);
low_int.notify();
wait(1, SC_SEC);
```

```
high_int.notify();
}
SC_CTOR(priority_interrupt) {
SC_THREAD(low_ISR);
SC_THREAD(high_ISR);
SC_THREAD(generator);
}
};
int sc_main(int argc, char* argv[]) {
priority_interrupt obj("Priority_INT");
sc_start(10, SC_SEC);
return 0;
}
```

testbench.cpp

```
// Code your testbench here.
// Uncomment the next line for SystemC modules.
// #include <systemc>
#include <systemc.h>
SC_MODULE(priority_interrupt) {
sc_event low_int, high_int;
void low_ISR() {
while(true) {
wait(low_int);
cout << "Low Priority Interrupt at "
<< sc_time_stamp() << endl;
wait(3, SC_SEC);
}
}
void high_ISR() {
while(true) {
wait(high_int);
cout << "High Priority Interrupt at "
<< sc_time_stamp() << endl;
wait(1, SC_SEC);
}
}
void generator() {
wait(1, SC_SEC);
```

Output:

Log Share

```
[2026-08-23 16:39:49 UTC] g++ -o sim -lsystemc *.cpp  && echo "Compile done. Starting run..
Compile done. Starting run...
Low Priority Interrupt at 1 s
High Priority Interrupt at 2 s

        SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
        Copyright (c) 1996-2018 by all Contributors,
        ALL RIGHTS RESERVED
Done
```